Lab 6 OOP

Lab Manual Object-Oriented Programming

Objective:

By the end of this lab, students will:
1. Understand **Multilevel** and **Hierarchical** Inheritance.
2. Learn **when to use constructors** and their impact on object initialization.
3. Implement **real-life scenarios** using **automatic (default) and parameterized constructors**.

➢ Automatic Constructor (Default Constructor)

What is it?

An **automatic constructor** is a constructor that is **provided by Java by default** (if no constructor is written manually). It doesn't take any parameters.

When to Use?

1. When you want to set **default values** inside the class.
2. When **every object has the same properties** and no specific input is required from the user.
3. Best for **standard or fixed scenarios**.

➢ Parameterized Constructor

What is it?

A **parameterized constructor** is a constructor that **takes input (arguments)** to initialize values.

When to Use?

1. When you want to **set custom values** at the time of object creation.
2. When **every object has different data**.
3. Best for **dynamic and user-specific scenarios**.

➢ Inheritance (Multilevel & Hierarchical)

Inheritance allows one class to acquire the properties and behavior of another class. It helps in code reusability and better organization of a program.

1. Multilevel Inheritance

Multilevel Inheritance is a type of inheritance where a class inherits from another class, and then another class inherits from it, forming a chain of inheritance.

Syntax of Multilevel Inheritance

```
// Parent Class
class A {
    // Properties and methods of class A
}

// Child Class (inherits from A)
class B extends A {
    // Additional properties and methods of class B
}
// Grandchild Class (inherits from B)
class C extends B {
    // Additional properties and methods of class C
}
// Main Class
public class Example {
    public static void main(String[] args) {
        C obj = new C();  // Creating object of the grandchild class
    }}
```

In Multilevel Inheritance:

Class C **inherits** from Class B, and Class B **inherits** from Class A.

This forms a **chain of inheritance** (Grandparent → Parent → Child)

Scenario 1: Multilevel Inheritance (Using Automatic Constructor)

Scenario:
A **Vehicle** has basic properties. A **Car** inherits from Vehicle, and an **ElectricCar** inherits from Car. We will use **automatic (default) constructors** so that inheritance works without explicitly defining constructors.

Code:

```
// Base class Vehicle
class Vehicle {
    String type = "General Vehicle";  // Default property
```

```
    void displayType() {
        System.out.println("Type: " + type);
    }
}
// Derived class Car (inherits from Vehicle)
class Car extends Vehicle {
    String brand = "Toyota";  // Default property
    void displayCar() {
        System.out.println("Brand: " + brand);
    }
}
// Derived class ElectricCar (inherits from Car)
class ElectricCar extends Car {
    int batteryLife = 300;  // Default property la leta hn km ma
    void displayElectricCar() {
        System.out.println("Battery Life: " + batteryLife + " km");
    }
}
//ya Main class
public class MultilevelExample {
    public static void main(String[] args) {
        // Object ha ElectricCar ka jo automatically get kra ga properties of
Vehicle and Car
        ElectricCar tesla = new ElectricCar();
        // Display krwa rha inherited properties
        tesla.displayType();  // ya Vehicle class
        tesla.displayCar();   // Ya Car class
        tesla.displayElectricCar(); // Ya ElectricCar class
    } }
```

Output:
Type: General Vehicle
Brand: Toyota
Battery Life: 300 km

Scenario 2: Multilevel Inheritance (Using Parameterized Constructor)

Scenario:
A **Company** has employees. A **Manager** inherits from Employee. A **ProjectManager** inherits from Manager. Since each employee has different details, we use **parameterized constructors**.

Code:

```
// Parent class Employee
class Employee {
    String name;
```

```
    double salary;

    // Parameterized constructor
    Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
    }
    void display() {
        System.out.println("Employee Name: " + name);
        System.out.println("Salary: $" + salary);
    }
}
// Child class Manager (inherits from Employee)
class Manager extends Employee {
    String department;
    // Constructor calling parent class constructor
    Manager(String name, double salary, String department) {
        super(name, salary);
        this.department = department;
    }
    void displayManager() {
        display();
        System.out.println("Department: " + department);
    }
}
// Grandchild class ProjectManager (inherits from Manager)
class ProjectManager extends Manager {
    String project;
    // Constructor calling parent class constructor
    ProjectManager(String name, double salary, String department, String
project) {
        super(name, salary, department);
        this.project = project;
    }
    void displayProjectManager() {
        displayManager();
        System.out.println("Project: " + project);
    }
}
// Main Class
public class CompanyExample {
    public static void main(String[] args) {
        // Creating object with parameterized constructor
        ProjectManager pm = new ProjectManager("Hassan", 90000, "IT", "AI
Research");
        // Display details
        pm.displayProjectManager();
    } }
```

Output:

Employee Name: Hassan
Salary: $90000.0
Department: IT
Project: AI Research

Scenario 3: Hierarchical Inheritance (Using Automatic Constructor)

Scenario:
A **Transport Service** has general details. **Car** and **Bike** inherit from Transport. Since vehicles have standard attributes, we use **automatic constructors**.

Code:

```
// Parent class Transport
class Transport {
    String type = "Public Transport";  // Default property

    void displayType() {
        System.out.println("Transport Type: " + type);
    }
}
// Child class Car (inherits from Transport)
class Car extends Transport {
    int seats = 4;  // Default property
    void displayCar() {
        System.out.println("Car Seats: " + seats);
    }
}
// Child class Bike (inherits from Transport)
class Bike extends Transport {
    boolean hasCarrier = true;  // Default property
    void displayBike() {
        System.out.println("Has Carrier: " + hasCarrier);
    }
}
// Main Class
public class TransportExample {
    public static void main(String[] args) {
        // Creating objects of child classes
        Car car1 = new Car();
        Bike bike1 = new Bike();
        // Displaying properties
        car1.displayType();
        car1.displayCar();
        bike1.displayType();
```

```
        bike1.displayBike();
    }
}
```

Output:

Transport Type: Public Transport
Car Seats: 4
Transport Type: Public Transport
Has Carrier: true

Scenario 4: Hierarchical Inheritance (Using Parameterized Constructor)

Scenario:
A **Hospital** has staff members. **Doctor** and **Nurse** inherit from Hospital. Since each staff member has different details, we use **parameterized constructors**.

Code:

```
// Parent class Hospital
class Hospital {
    String name;
    int experience;

    // Parameterized constructor
    Hospital(String name, int experience) {
        this.name = name;
        this.experience = experience;
    }
    void display() {
        System.out.println("Name: " + name);
        System.out.println("Experience: " + experience + " years");
    }
}
// Child class Doctor
class Doctor extends Hospital {
    String specialization;
    // Constructor calling parent class constructor
    Doctor(String name, int experience, String specialization) {
        super(name, experience);
        this.specialization = specialization;
    }
    void displayDoctor() {
        display();
        System.out.println("Specialization: " + specialization);
    }
}
```

```
// Child class Nurse
class Nurse extends Hospital {
    String department;
    // Constructor calling parent class constructor
    Nurse(String name, int experience, String department) {
        super(name, experience);
        this.department = department;
    }
    void displayNurse() {
        display();
        System.out.println("Department: " + department);
    }
}
// Main Class
public class HospitalExample {
    public static void main(String[] args) {
        Doctor doc = new Doctor("Dr. Ali", 10, "Cardiology");
        Nurse nurse = new Nurse("Nurse Sara", 5, "Pediatrics");
        doc.displayDoctor();
        nurse.displayNurse();
    }
}
```

Output:

Name: Dr. Ali
Experience: 10 years
Specialization: Cardiology
Name: Nurse Sara
Experience: 5 years
Department: Pediatrics